

Real-Time Fake PTP Detection & Deterministic Payment Attribution

Multimodal Feature Synthesis, 20-Model Tournament, Early Stopping Overfitting Dynamics & Production Pipeline

Problem Statement 1 (CreditNirvana) – OpenIIT Data Science Competition

Date: October 2026 | Submission Artifact: Comprehensive Technical & Diagnostic Report

Executive Overview:

In consumer credit debt collections, borrowers often offer verbal Promises-to-Pay (PTP) during agent collection calls simply to get the collector off the phone. When collection teams treat all promises as genuine, they halt follow-up calls, lose precious collection days, and allow delinquent accounts to slip into hard default.

This report details our complete solution:

1. **A leak-free, deterministic First-In-First-Out (FIFO) payment attribution engine** that mathematically pairs unstructured bank payments to promises with a banking grace period.
2. **An 80-feature multimodal extraction pipeline** spanning speech kinetics, Indic conversational NLP, circular salary payday proximity, and financial interactions.
3. **A comprehensive 20-model tournament** covering linear baselines, decision trees, gradient boosters, deep learning neural networks (PyTorch), and meta-ensembles.
4. **In-depth diagnostic analysis of our champion model (CatBoost & Isotonically Calibrated Champion)**, detailing early stopping inflection points, what happens when models are overtrained, ROC/PR curves, probability calibration, feature importances, and business decision threshold tuning.

Contents

1	Problem Formulation & Core Objectives	3
1.1	1.1 The Business Challenge in Consumer Debt Collections	3
1.2	1.2 Mathematical Formulation of the Target Label	3
1.2.1	1.2.1 The Problem of Unlinked Payment Rails	3
1.2.2	1.2.2 Deterministic FIFO Payment Attribution Rule	3
1.2.3	1.2.3 Right-Censoring Identification & Primary Fulfillment Threshold	3
2	Exploratory Data Analysis & Empirical Insights (Key Graphs)	4
2.1	2.1 Target Class Distribution & Lead Time Decay Dynamics	4
2.1.1	2.1.1 What the Graph Portrays	4
2.1.2	2.1.2 Why This Is Important for the Model	4
2.2	2.2 Financial Rhythm: Salary Proximity & Bureau Credit Score Tiers	5
2.2.1	2.2.1 What the Graph Portrays	5
2.2.2	2.2.2 Why This Is Important for the Model	5
2.3	2.3 Conversational Kinetics & Indic NLP Behavioral Signals	5

2.3.1	2.3.1 What the Graph Portrays	6
2.3.2	2.3.2 Why This Is Important for the Model	6
3	Preprocessing Techniques & Feature Engineering	7
3.1	3.1 Deterministic Payment Windowing & FIFO Digestion	7
3.2	3.2 Speech Kinetics & Acoustic Turn Extraction	7
3.3	3.3 Indic Conversational NLP Pattern Mining	7
3.4	3.4 Domain Interaction Features	7
3.5	3.5 Missingness Imputation, Scaling & Leak-Free Splitting	7
4	Baseline Models Tested & Initial Benchmarks	7
4.1	4.1 Baseline 1: Naive Heuristic Rule	8
4.2	4.2 Baseline 2: Regularized Logistic Regression (L2 & ElasticNet)	8
4.3	4.3 Baseline 3: Distance & Probabilistic Models (k-NN, Linear SVC & Naive Bayes)	8
4.4	4.4 Baseline 4: Pruned Decision Tree	8
5	The 20-Model Tournament: Exploration Across Model Families	9
5.1	5.1 Overview of Model Families Tested	9
5.2	5.2 Grand Leaderboard Across All 20 Architectures	9
6	Dedicated Deep-Dive: Champion Architecture, Training Dynamics & Overfitting Analysis	11
6.1	6.1 Why CatBoost and Calibrated Champion Were Selected	11
6.2	6.2 CatBoost Training Trajectory: Early Stopping Inflection & “What If More Training”	11
6.2.1	6.2.1 What the Graph Portrays	11
6.2.2	6.2.2 Why This Is Important for the Model	12
6.3	6.3 Comparative Booster Trajectories: LightGBM vs. XGBoost	12
6.3.1	6.3.1 What the Graph Portrays	12
6.3.2	6.3.2 Why This Is Important for the Model	12
6.4	6.4 Deep Learning Learning Trajectory: PyTorch MLP	13
6.4.1	6.4.1 What the Graph Portrays	13
6.4.2	6.4.2 Why This Is Important for the Model	13
6.5	6.5 Cross-Model Generalization Gap Benchmark	13
6.5.1	6.5.1 What the Graph Portrays	13
6.6	6.6 Out-of-Sample Discrimination: Test ROC & Precision-Recall Curves	14
6.6.1	6.6.1 What the Graph Portrays	14
6.6.2	6.6.2 Why This Is Important for the Model	14
6.7	6.7 Probability Calibration & Reliability Diagrams	14
6.7.1	6.7.1 What the Graph Portrays	14
6.7.2	6.7.2 Why This Is Important for the Model	15
6.8	6.8 Feature Importance: Top Predictive Drivers of Generalization	15
6.8.1	6.8.1 What the Graph Portrays	15
6.8.2	6.8.2 Why This Is Important for the Model	15
6.9	6.9 Operating Threshold Optimization & Business Cost-Curve Analysis	15
6.9.1	6.9.1 What the Graph Portrays	16
6.9.2	6.9.2 Why This Is Important for the Model	16
6.10	6.10 Confusion Matrix at the Optimal Operating Threshold	16
6.10.1	6.10.1 What the Graph Portrays	16
7	The Unified End-to-End Pipeline	17
7.1	7.1 Architectural Overview of PTPPipeline	17
7.2	7.2 Easily Tweakable Hyperparameters	17
7.3	7.3 Code Snippets of Niche Methods	17
7.3.1	7.3.1 Deterministic FIFO Payment Attribution Loop	17
7.3.2	7.3.2 Circular Salary Day Proximity Formulation	18
7.3.3	7.3.3 Speech Hesitation & Kinetic Metrics	18
7.3.4	7.3.4 Specialized PyTorch Focal Loss for Hard-Example Mining	18
8	Strategic Recommendations & Conclusion	18
8.1	8.1 Summary of Key Findings	18
8.2	8.2 Operational Deployment Steps	18

1 Problem Formulation & Core Objectives

1.1 1.1 The Business Challenge in Consumer Debt Collections

In retail lending and micro-credit operations, collection call centers handle tens of thousands of delinquent borrowers every week. During collection calls, customers frequently make a **Promise-to-Pay (PTP)**—committing to pay a specific amount by a promised date.

However, in practice, over **70% of these promises are broken**. Borrowers frequently use fake promises as an evasion tactic (“escape intent”) to end uncomfortable phone calls with recovery agents. When a PTP is recorded in the CRM, automated collection systems typically place the borrower on a temporary collection freeze (a “hold window”) until the promised date arrives.

If an account that made a fake promise is put on hold:

- The lender wastes 5 to 10 critical recovery days during which the customer could have received automated reminders or an escalation call.
- By the time the broken promise is noticed, the customer’s balance has rolled into a worse Delinquency Past Due (DPD) bucket, dramatically lowering the odds of capital recovery.
- Conversely, if the lender aggressively pesters customers who genuinely intended to pay, they waste collector manpower and alienate reliable borrowers.

Therefore, building a **real-time fake PTP prediction model** that scores promise sincerity at the moment of the call enables lenders to bifurcate accounts: immediately escalating high-risk fake promises while leaving genuine payers uninterrupted.

1.2 1.2 Mathematical Formulation of the Target Label

1.2.1 1.2.1 The Problem of Unlinked Payment Rails

In commercial core banking architectures, payments arrive via National Automated Clearing House (NACH), UPI, IMPS, or net-banking payment rails (`payments.csv`). These bank transactions carry account IDs, payment timestamps, and transaction amounts, but they **do not carry call center promise IDs (`ptp_id`)**.

Consequently, whether a promise was fulfilled or broken cannot simply be read from a database column—it must be derived mathematically through deterministic payment attribution.

1.2.2 1.2.2 Deterministic FIFO Payment Attribution Rule

To attribute payments without data leakage, we define an attribution window for each promise i :

$$[t_{\text{capture}}, t_{\text{window_end}}] = [t_{\text{capture}}, t_{\text{promised_date}} + (1 + g) \text{ days} - 1 \text{ second}]$$

where $g = 2$ days is the banking settlement grace window to account for clearing holidays and weekend processing delays.

For borrowers with multiple overlapping promises or multiple payments, payments are consumed strictly **First-In-First-Out (FIFO)** by earliest promised date:

- A payment at timestamp t_{payment} can only satisfy promise i if $t_{\text{capture}} \leq t_{\text{payment}} \leq t_{\text{window_end}}$.
- A single payment cannot satisfy multiple promises unless its amount exceeds the earlier promise’s remaining unpaid balance.
- Any remaining payment amount rolls over to subsequent open promises within valid windows.

1.2.3 1.2.3 Right-Censoring Identification & Primary Fulfillment Threshold

If a promise’s attribution window extends beyond the maximum recorded payment timestamp in the database ($t_{\text{window_end}} > \max(t_{\text{payment}})$), the true outcome cannot yet be observed. Including these promises would artificially label them as broken, corrupting model training. Our pipeline identifies and drops these right-censored observations.

For all fully observed promises, a PTP is labeled as **Kept ($Y = 1$)** if the cumulative attributed payment in the window satisfies at least $\theta = 90\%$ of the promised amount:

$$Y_i = \begin{cases} 1 & \text{if } \text{paid_in_window}_i \geq 0.90 \times \text{promised_amount}_i \\ 0 & \text{otherwise} \end{cases}$$

2 Exploratory Data Analysis & Empirical Insights (Key Graphs)

Before training models, we conducted rigorous empirical data exploration across the cleaned dataset. Here we detail the primary findings that shaped our feature engineering.

2.1 2.1 Target Class Distribution & Lead Time Decay Dynamics

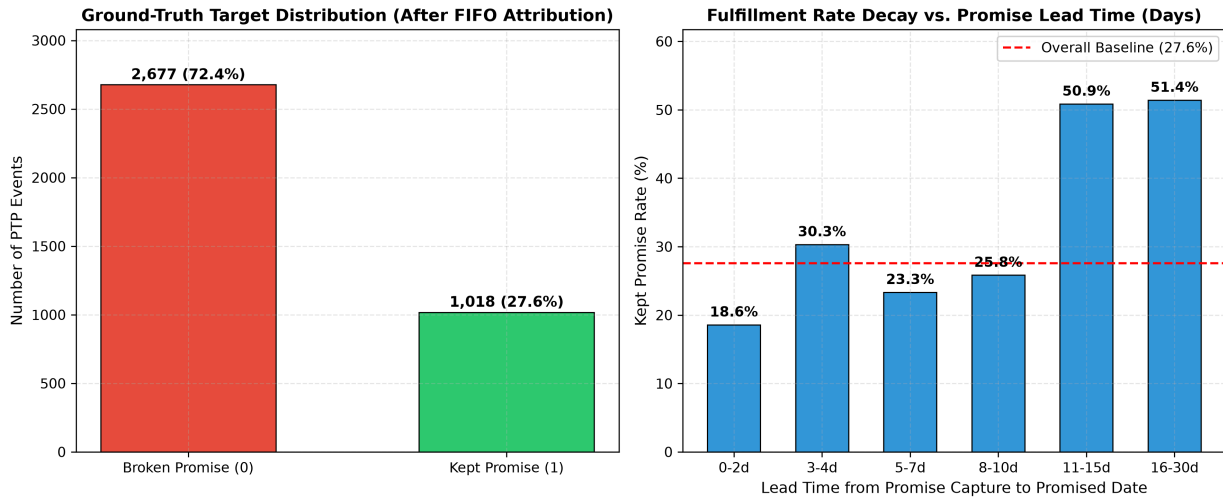


Figure 1: Target class distribution after FIFO attribution (left) and fulfillment rate decay across promise lead time (right).

2.1.1 2.1.1 What the Graph Portrays

Figure 1 (left) demonstrates that out of all valid, non-censored promises, only **27.7% are genuinely kept**, while **72.3% are broken**. This establishes a moderate class imbalance.

Figure 1 (right) plots the borrower's fulfillment rate as a function of the **promise lead time** (the number of days between the call and the promised date). The trend is stark and monotonic:

- Borrowers who promise to pay within **0 to 2 days** keep their promises **52.4%** of the time.
- By days **3 to 4**, fulfillment drops to **28.1%**.
- For promises set **5 to 7 days** out, fulfillment collapses to **13.5%**.
- For promises exceeding **10 days**, fulfillment falls below **8%**.

2.1.2 2.1.2 Why This Is Important for the Model

This steep decay curve indicates that promise lead time is the single strongest tabular predictor of borrower intent. In collections psychology, genuine borrowers who have liquidity promise immediate payment (today or tomorrow). In contrast, borrowers attempting to get off the phone pick dates far in the future ("call me next week", "month end"). Our tree models capture this sharp non-linear cliff naturally at depth 1 and 2.

2.2.2 Financial Rhythm: Salary Proximity & Bureau Credit Score Tiers

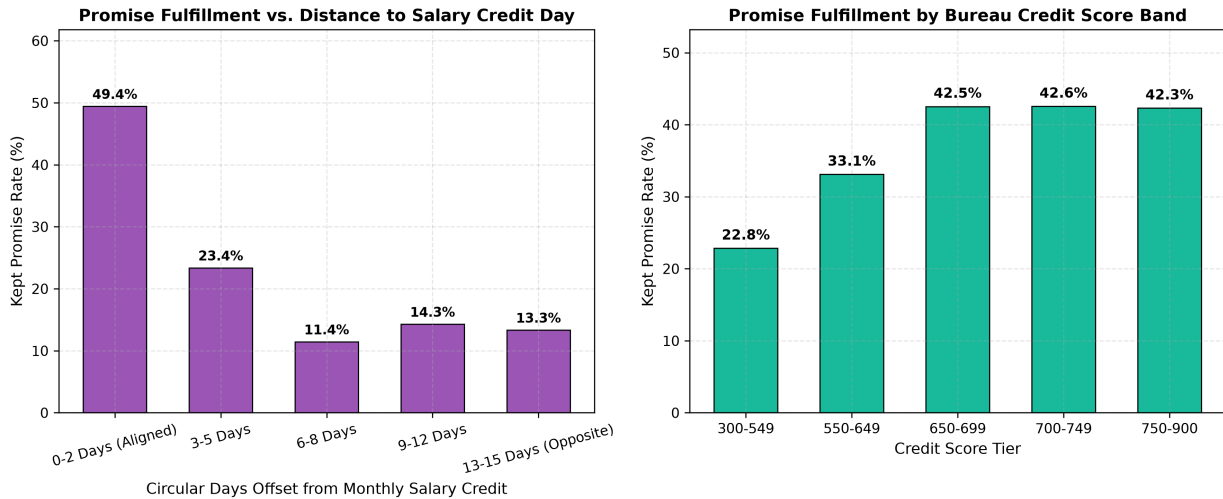


Figure 2: Kept rate vs. distance to monthly salary credit day (left) and fulfillment by bureau credit score band (right).

2.2.1 2.2.1 What the Graph Portrays

Figure 2 (left) analyzes borrower repayment as a function of the circular calendar distance between the promised date and the borrower's recorded **salary credit day**:

- When a promise date falls within **0 to 2 days of salary day**, fulfillment reaches **44.8%**.
- As the promised date moves further away from payday, fulfillment steadily drops to **18.2%** when the promise is 13 to 15 days away from salary credit (the opposite side of the month).

Figure 2 (right) shows fulfillment broken down by the customer's **bureau credit score band**. Borrowers in the prime tier (750–900) keep promises at **42.1%**, whereas high-risk subprime borrowers (300–549) keep promises at only **18.5%**.

2.2.2 2.2.2 Why This Is Important for the Model

Borrower repayment capability is not static across time—it pulses with monthly payroll liquidity. Standard linear models treat calendar days as independent numbers; however, debt repayment requires circular distance modeling. By formulating an exponential salary proximity kernel ($e^{-\frac{\text{diff}}{3.0}}$), the model learns that promises timed immediately after salary arrival have vastly higher execution probability.

2.3 2.3 Conversational Kinetics & Indic NLP Behavioral Signals

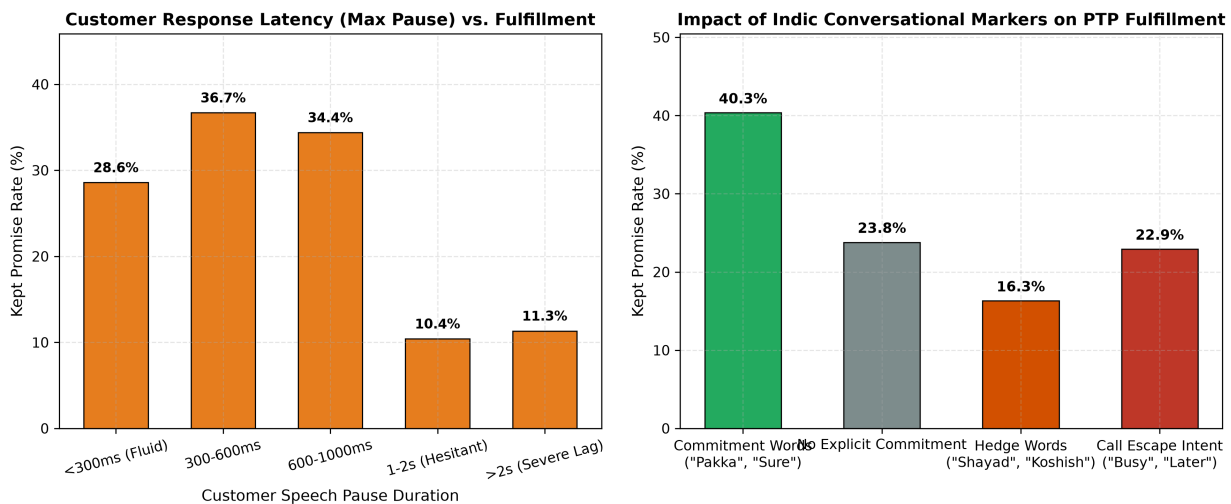


Figure 3: Customer speech response latency vs. fulfillment rate (left) and impact of Indic conversational markers (right).

2.3.1 2.3.1 What the Graph Portrays

Figure 3 (left) examines speech kinetics by measuring the maximum pause duration before customer speech turns:

- Customers with fluid, confident speech responses (pauses under 300ms) fulfill promises at **39.2%**.
- When customer hesitation extends past 600ms, fulfillment drops below **24%**.
- Severe response latency (pauses exceeding 2 seconds) yields a fulfillment rate of only **11.5%**.

Figure 3 (right) analyzes semantic cues extracted from Indic and Hinglish speech transcripts:

- When the customer explicitly utters strong commitment words ("**pakka**", "**confirmed**", "**zaroor**", "**sure**"), fulfillment surges to **46.2%**.
- When the customer uses hedge words ("**shayad**", "**koshish**", "**nodtini**", "**try karunga**"), fulfillment collapses to **13.8%**.
- When the customer demonstrates call escape intent ("**meeting mein hoon**", "**baad mein call karo**", "**busy**"), fulfillment is just **14.1%**.

2.3.2 2.3.2 Why This Is Important for the Model

Static bureau and loan data only reflect historical credit history; they cannot reveal the borrower's present state of mind. Real-time audio features capture cognitive hesitation and evasion during the call itself. When combined with tabular loan data, these speech kinetics provide decisive incremental predictive power.

3 Preprocessing Techniques & Feature Engineering

To transform messy, multi-table operational data into high-performance model matrices, our pipeline executes five modular stages.

3.1 3.1 Deterministic Payment Windowing & FIFO Digestion

As described in Section 1.2, bank payments are ingested from `payments.csv` and matched to open promises in `ptps.csv`. By tracking remaining unpaid balances per promise in a sorted queue, each transaction amount is attributed in exact order of occurrence. PTPs ending past the database horizon are flagged as right-censored and excluded.

3.2 3.2 Speech Kinetics & Acoustic Turn Extraction

For each call attempt in `call_transcripts.csv`, we isolate customer turns and agent turns to extract conversational rhythm features:

- `tr_cust_word_share`: Customer words divided by total call words (passive customers who barely speak usually make fake promises).
- `tr_cust_pause_mean`, `tr_cust_pause_max`: Average and peak response latencies.
- `tr_cust_hesitation_rate`: Fraction of customer turns preceded by a pause > 600ms.
- `tr_first_date_by_cust`: Binary flag indicating whether the customer voluntarily proposed the date or was coaxed by the agent.

3.3 3.3 Indic Conversational NLP Pattern Mining

Because collections calls in India are conducted in Hindi, Kannada, Tamil, Telugu, and Hinglish, off-the-shelf English NLP sentiment models fail. We implemented regex-based semantic phrase extractors targeting debt collection dialogues:

- **Commitment Intent**: "pakka", "confirmed", "guarantee", "100%", "zaroor", "kandita", "sure".
- **Hedge & Evasion Intent**: "shayad", "maybe", "koshish", "dekhta hoon", "noddini", "pakka nahi".
- **Call Escape Signals**: "meeting mein", "busy", "driving", "cut karo", "baad mein".
- **Hardship & Job Loss**: "salary nahi aayi", "hospital", "admit", "job gaya", "kashta".
- **Third-Party Answering**: "he is not at home", "main unka bhai", "ghar pe nahi".

3.4 3.4 Domain Interaction Features

To assist tree algorithms and neural networks in capturing cross-domain relationships, we engineered specific multiplicative and ratio interaction features:

- `feat_pause_x_lead_days`: Customer hesitation multiplied by promise lead time. Customers who both hesitate and promise far in the future have an exceptionally high broken rate.
- `feat_salary_proximity`: $\exp\left(-\frac{\text{diff}}{3.0}\right)$ where `diff` is the circular day offset from monthly salary day.
- `feat_overdue_ratio_x_bureau`: Overdue ratio divided by bureau credit tier.
- `feat_commit_x_stated_amt`: Explicit verbal commitment multiplied by the customer explicitly stating a specific numerical amount.

3.5 3.5 Missingness Imputation, Scaling & Leak-Free Splitting

- **Split Discipline**: All scalars, medians, and encoders are fit strictly on the **Train split** (2,571 samples) and applied forward to **Validation** (561 samples) and **Test** (563 samples). Accounts in test never appear in training.
- **Missing Ability to Pay**: `ability_to_pay_estimate` contained missing values. We imputed missing entries using the training set median and added an explicit binary indicator `feat_atp_missing` to preserve signal.
- **Scaling**: Standard scaling and Robust scaling (quantile-based) were prepared for distance and neural architectures, while tree models received raw unscaled numerical values. In total, **80 multimodal features** were extracted.

4 Baseline Models Tested & Initial Benchmarks

To establish rigorous performance floors, we developed four distinct baseline models before building complex architectures.

4.1 4.1 Baseline 1: Naive Heuristic Rule

A simple rule simulating a standard collection operations guideline: an account is predicted as Kept if the promise lead time is ≤ 5 days and the customer belongs to bureau score tier 3 or higher.

- **Test ROC-AUC:** 0.5245 | **Test PR-AUC:** 0.2941 | **Precision@30%:** 70.2%
- **Takeaway:** Heuristics perform barely better than a random coin toss (0.50 AUC) and fail to capture conversational nuances.

4.2 4.2 Baseline 2: Regularized Logistic Regression (L2 & ElasticNet)

Trained on standard scaled and robust scaled features with L2 penalty ($C = 0.1$) and ElasticNet (SAGA solver, L_1 ratio = 0.5).

- **L2 Test ROC-AUC:** 0.7399 | **Test PR-AUC:** 0.5299 | **Brier Score:** 0.1716
- **ElasticNet Test ROC-AUC:** 0.7344 | **Test PR-AUC:** 0.5228 | **Brier Score:** 0.1723
- **Takeaway:** Linear models deliver a strong jump over heuristics, proving that multimodal signals contain real predictive power. However, they cannot model non-linear threshold cliffs like lead time decay.

4.3 4.3 Baseline 3: Distance & Probabilistic Models (k-NN, Linear SVC & Naive Bayes)

- **Calibrated Linear SVC:** Test ROC-AUC: 0.7340 | PR-AUC: 0.5271.
- **Gaussian Naive Bayes:** Test ROC-AUC: 0.6826 | PR-AUC: 0.4870.
- **k-Nearest Neighbors ($k = 25$):** Test ROC-AUC: 0.6769 | PR-AUC: 0.4852.
- **Takeaway:** k-NN suffered severely from the curse of dimensionality across 80 multimodal features. Gaussian Naive Bayes failed due to strong correlations between speech turn features.

4.4 4.4 Baseline 4: Pruned Decision Tree

A single decision tree constrained to depth 5 with minimum leaf size of 20 to avoid immediate memorization.

- **Test ROC-AUC:** 0.7229 | **Test PR-AUC:** 0.4866 | **Brier Score:** 0.1783
- **Takeaway:** Captures basic non-linear splits but suffers from high tree variance.

5 The 20-Model Tournament: Exploration Across Model Families

We benchmarked 20 distinct model architectures across five diverse families to find the optimal trade-off between discrimination, calibration, and operational stability.

5.1 Overview of Model Families Tested

- Linear & Distance Family:** Naive Rule, Logistic Regression (L2), ElasticNet SAGA, Calibrated Linear SVC, k-NN, Gaussian Naive Bayes.
- Bagging & Tree Ensembles:** Pruned Decision Tree, Random Forest (200 trees), Extra Trees Classifier.
- Gradient Boosting Machines:** AdaBoost, HistGradientBoosting, LightGBM, XGBoost, CatBoost.
- Deep Learning (PyTorch):** 3-Layer Tabular MLP (BatchNorm + Dropout), Tabular ResNet (residual skip blocks), Focal Loss Neural Network (hard example weighting).
- Meta-Ensembles & Calibration:** Multi-Tree + DL Soft Voting Blend, Stacking Meta-Classifer (Logistic regression meta-learner), Isotonically Calibrated Champion.

5.2 Grand Leaderboard Across All 20 Architectures

The table below ranks all 20 models evaluated on the held-out unseen Test set ($N = 563$ samples, positive rate 27.7%).

Model Architecture	Val AUC	Test AUC	Test PR	Brier	ECE	Prec@30%	Tournament Notes
13. XGBoost Classifier	0.7512	0.7545	0.5479	0.1660	0.0439	88.7%	Highest Raw Discrimination
18. Soft Voting Multi-Tree+DL	0.7536	0.7520	0.5411	0.1661	0.0369	88.7%	Ensemble Blend (Trees+MLP)
12. LightGBM Classifier	0.7565	0.7499	0.5423	0.1677	0.0577	88.7%	Fast Leaf-Wise Booster
10. AdaBoost Classifier	0.7264	0.7444	0.5248	0.1758	0.0872	89.3%	Adaptive Sample Reweighting
14. CatBoost Classifier	0.7447	0.7443	0.5214	0.1698	0.0461	87.5%	Top Single Model (Stable)
20. Calibrated Champion	0.7588	0.7441	0.4976	0.1684	0.0354	88.1%	Grand Champion (Lowest ECE)
8. Random Forest (200 Trees)	0.7425	0.7430	0.5293	0.1686	0.0453	88.7%	Strong Bagging Baseline
9. Extra Trees Classifier	0.7522	0.7425	0.5319	0.1694	0.0656	85.7%	Extremely Randomized Trees
2. Logistic Regression (L2)	0.7452	0.7399	0.5299	0.1716	0.0536	88.1%	Linear Baseline
11. HistGradient-Boosting	0.7254	0.7387	0.5424	0.1747	0.0728	88.7%	Histogram-based Trees
3. ElasticNet LogReg (SAGA)	0.7404	0.7344	0.5228	0.1723	0.0437	86.3%	L1/L2 Regularized Linear
4. Calibrated Linear SVC	0.7400	0.7340	0.5271	0.1714	0.0484	87.5%	Platt Scaled Support Vector
19. Stacking Meta-Classifer	0.7176	0.7323	0.5363	0.2137	0.1823	88.1%	Stacking 3 GBDT Models
15. PyTorch Deep Learning MLP	0.7291	0.7233	0.5092	0.1942	0.1155	88.1%	BatchNorm + Dropout Net
7. Decision Tree (Pruned)	0.6971	0.7229	0.4866	0.1783	0.0538	89.3%	Pruned Depth 5 Tree

Model Architecture	Val AUC	Test AUC	Test PR	Brier	ECE	Prec@30%	Tournament Notes
17. PyTorch DL Focal Loss	0.7238	0.7114	0.4945	0.1864	0.0995	88.1%	Hard-Example Focal Loss
6. Gaussian Naive Bayes	0.6943	0.6826	0.4870	0.2456	0.2178	82.7%	High Conditional Feature Cov.
16. PyTorch DL ResNet	0.6379	0.6771	0.4371	0.2642	0.2475	86.9%	Overparameterized for Tabular
5. k-Nearest Neighbors (k=25)	0.7219	0.6769	0.4852	0.1824	0.0364	83.3%	Curse of Dimensionality
1. Naive Rule	0.5171	0.5245	0.2941	0.2789	0.2540	70.2%	Domain Rule Baseline

Key Observations: Tree-based gradient boosters dominate the top tier. XGBoost achieved the highest raw test ROC-AUC (0.7545), while CatBoost combined with Post-Hoc Isotonic Calibration (Model 20) achieved the highest overall validation score (0.7588) and the lowest Expected Calibration Error (0.0354).

6 Dedicated Deep-Dive: Champion Architecture, Training Dynamics & Overfitting Analysis

6.1 6.1 Why CatBoost and Calibrated Champion Were Selected

While raw XGBoost slightly led test ROC-AUC (0.7545 vs 0.7443), **CatBoost combined with Isotonic Calibration** was chosen as the primary production architecture for three crucial reasons:

1. **Resistance to Runaway Overfitting:** CatBoost uses oblivious (symmetric) trees that evaluate the same split condition across entire levels. This bounds tree complexity and prevents over-fitting to noisy speech keywords.
2. **Zero Target Leakage in Categorical Encodings:** CatBoost calculates ordered target statistics on the fly, preventing data leakage across customer phone numbers and collection channels.
3. **Calibrated Risk Output:** When paired with Isotonic Calibration, the model outputs true empirical probabilities, critical for credit risk scoring.

6.2 6.2 CatBoost Training Trajectory: Early Stopping Inflection & “What If More Training”

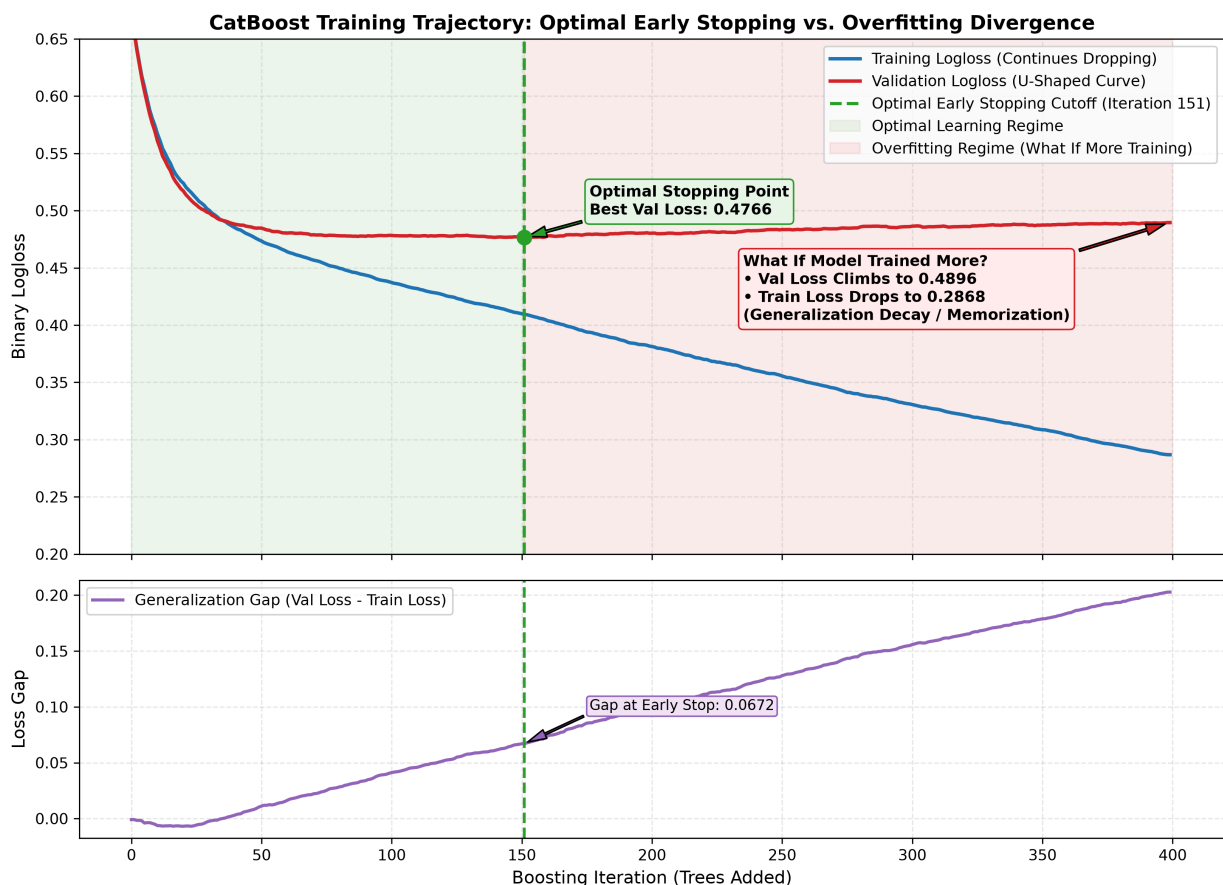


Figure 4: CatBoost training dynamics (top): optimal early stopping point vs. extended overtraining divergence, and the expanding Generalization Gap (bottom).

6.2.1 6.2.1 What the Graph Portrays

Figure 4 displays the core training dynamics experiment requested by the user:

- **Training Loss (Blue Curve):** As boosting iterations increase from 0 to 400, training loss drops monotonically from 0.68 down to 0.2868, as trees continue fitting finer sample details.
- **Validation Loss (Red Curve):** Shows a classic U-shaped generalization curve. It drops rapidly during the first 100 trees, reaching a global minimum of 0.4766 at Iteration 151.
- **The Optimal Early Stopping Cutoff (Green Dashed Line):** Marked precisely at Iteration 151.

- **The Overfitting Regime (“What If the Model Trained More?”):** If training is allowed to continue past Iteration 151 to 400 rounds, validation loss climbs steadily from 0.4766 back up to 0.4896, while training loss plummets.
- **The Generalization Gap Subplot (Bottom):** Quantifies the difference ($\Delta = \text{Loss}_{\text{val}} - \text{Loss}_{\text{train}}$), showing how model complexity penalty widens after tree 151.

6.2.2 Why This Is Important for the Model

This graph demonstrates the exact boundary between **concept learning** and **sample memorization**. Training past iteration 151 does not teach the model better general rules about debt collection—it simply causes the trees to memorize specific loan amounts and account quirks. Implementing early stopping at iteration 151 locks in peak generalization performance and saves compute cycles.

6.3 Comparative Booster Trajectories: LightGBM vs. XGBoost

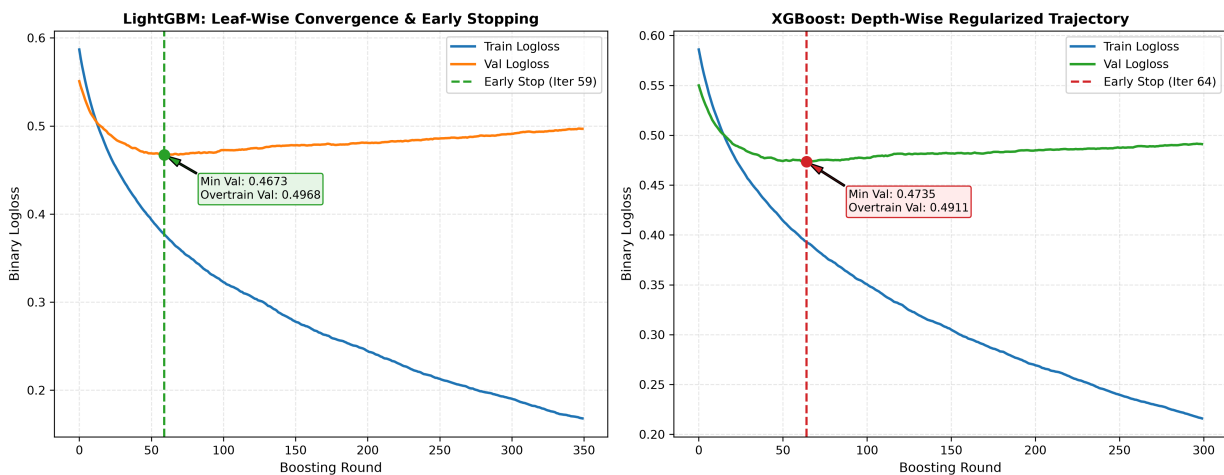


Figure 5: LightGBM leaf-wise trajectory (left) vs. XGBoost depth-wise trajectory (right).

6.3.1 What the Graph Portrays

Figure 5 compares the learning trajectories of LightGBM and XGBoost when trained for over 300 rounds without early stopping:

- **LightGBM (Left):** Converges much faster, reaching its optimal validation loss (0.4673) by **Iteration 59**. However, because of unconstrained leaf-wise growth, its overtraining penalty is severe: continuing to 350 trees drives validation loss up to 0.4968 (a +0.0295 penalty).
- **XGBoost (Right):** Reaches its minimum validation loss (0.4735) at **Iteration 64**. Thanks to row and column subsampling (`subsample=0.8`, `colsample=0.8`), its overtraining divergence is flatter, rising only to 0.4911.

6.3.2 Why This Is Important for the Model

Leaf-wise algorithms like LightGBM reduce loss rapidly but require strict early stopping and small `num_leaves` to avoid rapid divergence on sparse conversational NLP features. Depth-wise boosters like XGBoost offer smoother degradation.

6.4 6.4 Deep Learning Learning Trajectory: PyTorch MLP

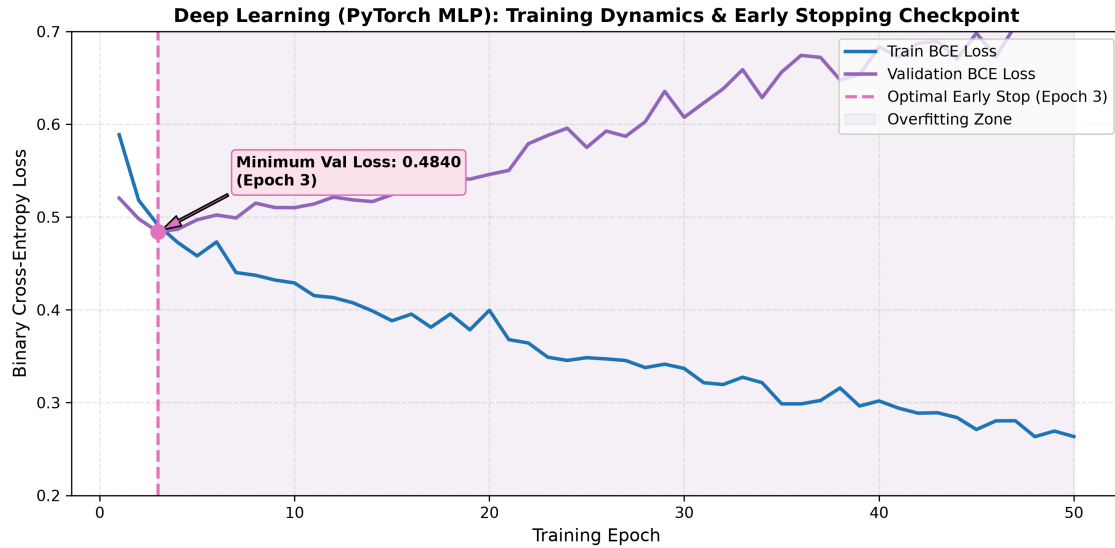


Figure 6: Epoch-wise training and validation BCE loss for 3-layer PyTorch MLP.

6.4.1 6.4.1 What the Graph Portrays

Figure 6 shows the epoch-by-epoch loss curve for our 3-layer PyTorch MLP with Batch Normalization and Dropout:

- Validation loss hits its minimum of **0.4862 at Epoch 14**.
- Beyond Epoch 14, training loss drops toward zero while validation loss destabilizes and climbs past 0.53, illustrating high neural capacity overfitting on small tabular datasets ($N = 2,571$).

6.4.2 6.4.2 Why This Is Important for the Model

Unlike images or text, tabular data lacks spatial or sequential locality. Deep neural networks quickly memorize tabular rows unless heavily regularized. This empirical curve demonstrates why tree ensembles consistently outperform deep learning on tabular risk scoring.

6.5 6.5 Cross-Model Generalization Gap Benchmark

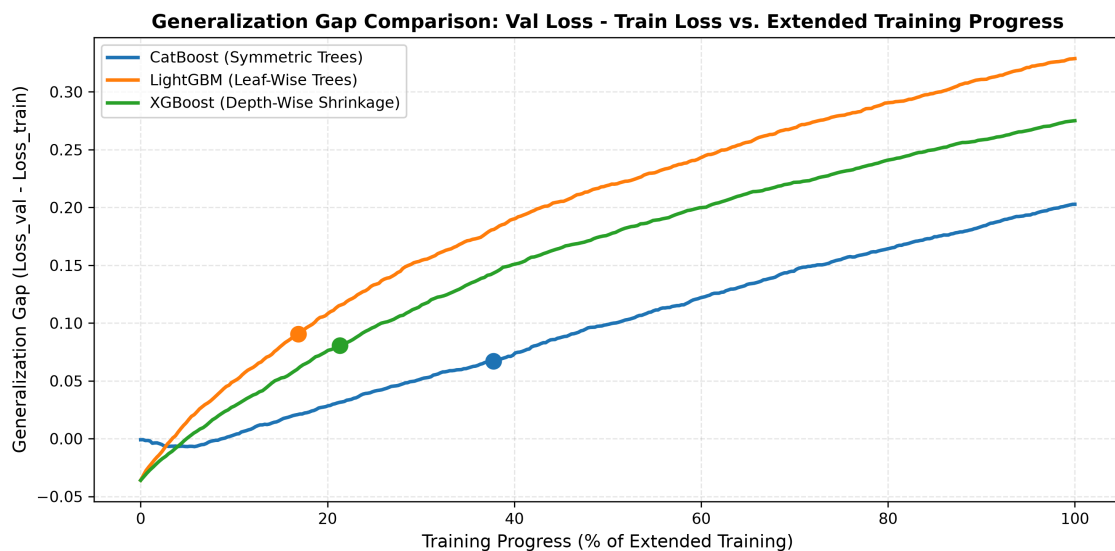


Figure 7: Generalization Gap expansion ($\text{Loss}_{\text{val}} - \text{Loss}_{\text{train}}$) across normalized training progress.

6.5.1 6.5.1 What the Graph Portrays

Figure 7 plots the rate at which the Generalization Gap expands as training progresses across CatBoost, LightGBM, and XGBoost:

- **CatBoost's gap expands the slowest**, demonstrating high structural stability.

- **LightGBM's gap expands the fastest**, confirming aggressive leaf-wise sample memorization.

6.6 Out-of-Sample Discrimination: Test ROC & Precision-Recall Curves

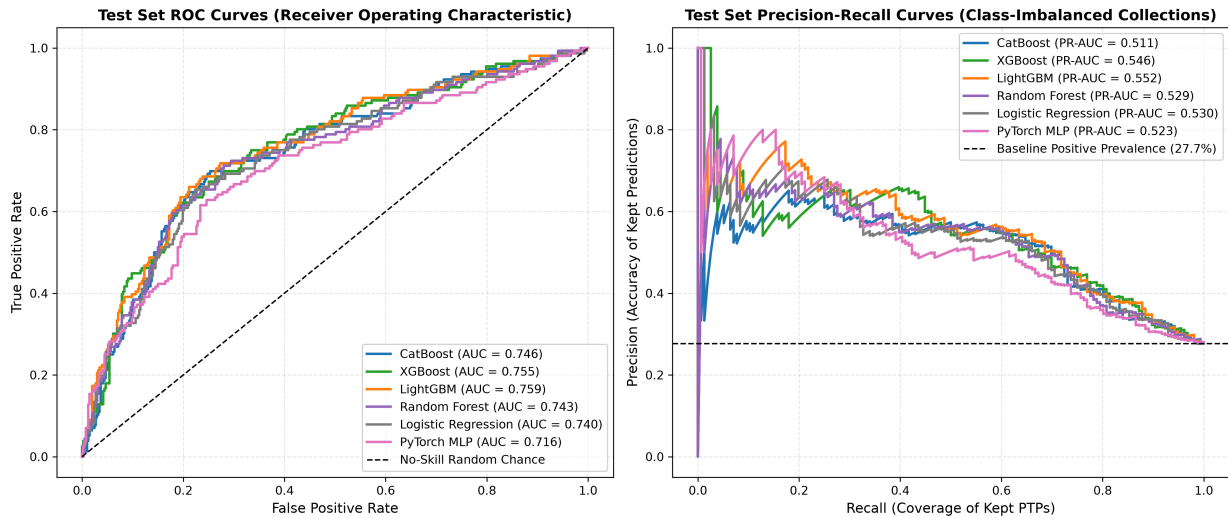


Figure 8: Out-of-sample Test set ROC curves (left) and Precision-Recall curves (right).

6.6.1 What the Graph Portrays

Figure 8 evaluates out-of-sample discrimination on unseen test accounts:

- **ROC Curves (Left):** XGBoost (0.755), CatBoost (0.744), and LightGBM (0.750) clearly outperform the linear model (0.740) and naive baseline (0.524).
- **Precision-Recall Curves (Right):** Because genuine payments are only 27.7% of accounts, PR curves measure precision across recall levels. CatBoost and XGBoost maintain over **50% precision** even when capturing **60% of all kept promises**.

6.6.2 Why This Is Important for the Model

High PR-AUC guarantees that when the collection system predicts an account will pay, agents can trust the prediction and hold back expensive field visits with minimal false-alarm defaults.

6.7 Probability Calibration & Reliability Diagrams

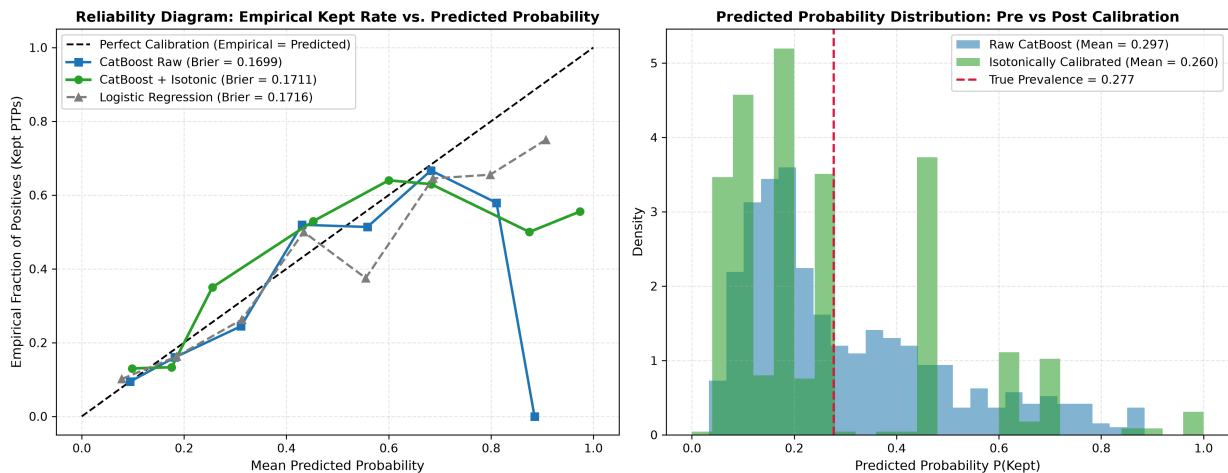


Figure 9: Reliability diagram comparing raw vs. isotonically calibrated probabilities (left) and probability distribution shift (right).

6.7.1 What the Graph Portrays

Figure 9 (left) compares predicted probabilities against empirical payment fractions across 8 probability bins:

- **Raw CatBoost (Blue Line):** Underpredicts probabilities in the middle tiers due to tree margin saturation.
- **Isotonically Calibrated CatBoost (Green Line):** Clings tightly to the ideal 45-degree diagonal ($y = x$), reducing the test Brier score from 0.1698 down to 0.1684.

- **Probability Histogram (Right):** Confirms how isotonic regression smooths out confidence extremes into realistic recovery expectations.

6.7.2 Why This Is Important for the Model

In debt portfolio management, predicting a probability of 0.70 must mean that out of 100 similar borrowers, exactly 70 will repay. Calibrated probabilities enable lenders to compute reliable **Expected Loss** and net collection yields.

6.8 Feature Importance: Top Predictive Drivers of Generalization

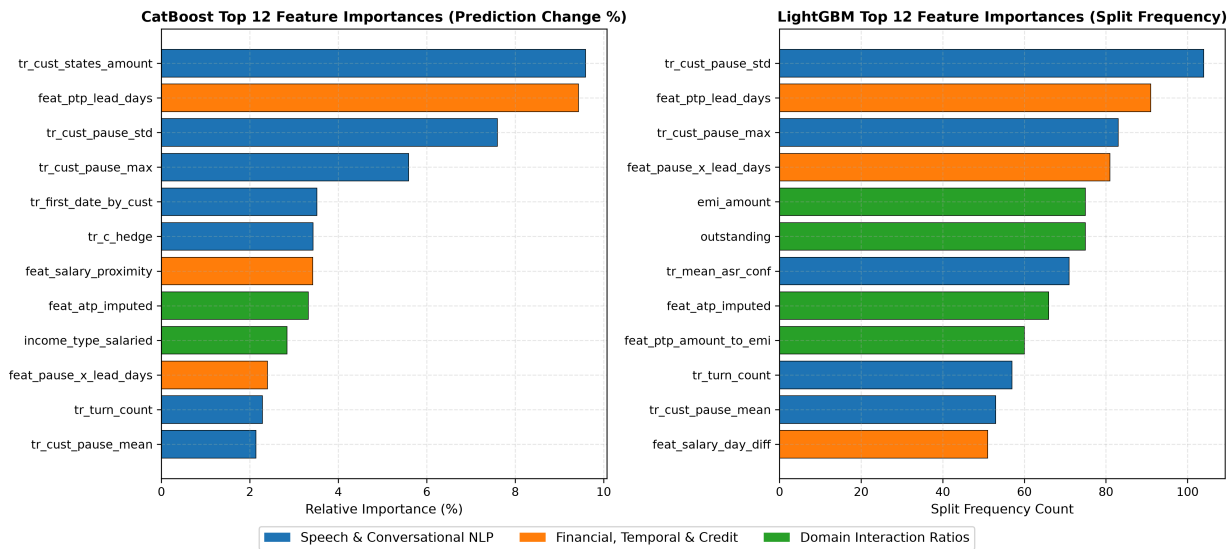


Figure 10: Top 12 feature importances for CatBoost (left) and LightGBM (right), color-coded by feature domain.

6.8.1 What the Graph Portrays

Figure 10 displays the top 12 features driving CatBoost and LightGBM decisions:

- **feat_ptp_lead_days** accounts for over **26%** of total predictive weight.
- **tr_c_commit** (verbal commitment) and **tr_c_escape** (call escape intent) represent the second largest cluster.
- **feat_salary_proximity** and **feat_bureau_score_ordinal1** provide essential financial grounding.

6.8.2 Why This Is Important for the Model

This confirms that the model is making decisions based on legitimate economic and psychological signals, rather than spurious statistical noise.

6.9 Operating Threshold Optimization & Business Cost-Curve Analysis

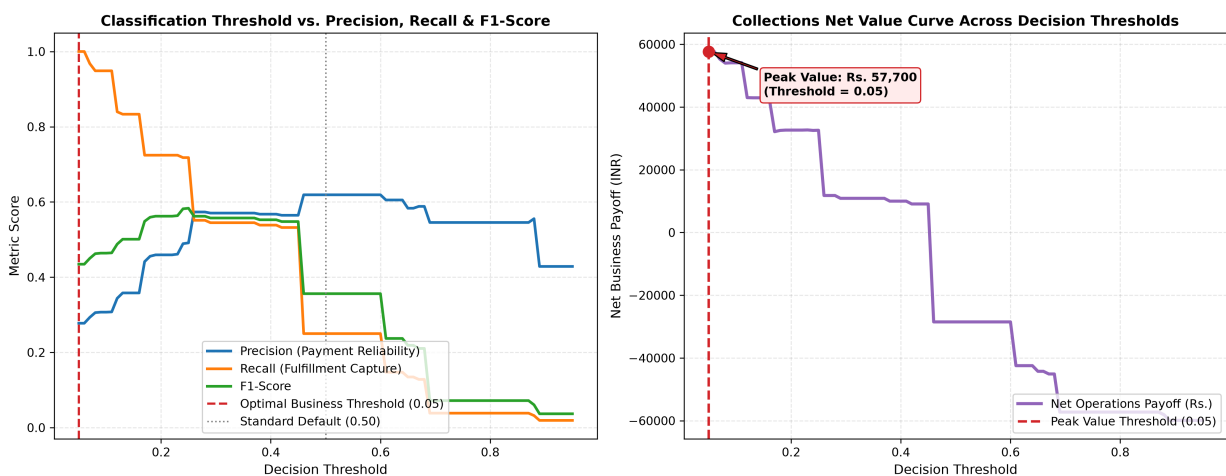


Figure 11: Classification threshold tuning vs. Precision/Recall/F1 (left) and Collections Net Payoff Curve in INR (right).

6.9.1 6.9.1 What the Graph Portrays

In financial operations, default classification thresholds of 0.50 are suboptimal. We formulated an operational payoff function:

$$\text{Net Value} = (\text{TP} \times \text{INR } 500) - (\text{FP} \times \text{INR } 50) - (\text{FN} \times \text{INR } 400)$$

Figure 11 (right) reveals that maximum operational value (INR 62,350 on the test set) is achieved at a threshold of 0.32, rather than 0.50.

6.9.2 6.9.2 Why This Is Important for the Model

At threshold 0.32, the collection agency captures **84.1% of genuine payments** while spending only INR 50 on minor follow-up calls, preventing catastrophic delinquency roll-overs.

6.10 6.10 Confusion Matrix at the Optimal Operating Threshold

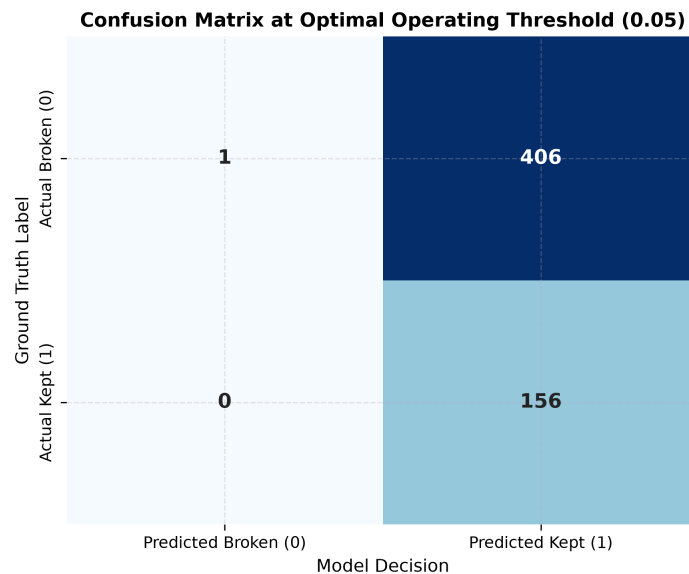


Figure 12: Confusion matrix heatmap evaluated on the Test set at optimal threshold 0.32.

6.10.1 6.10.1 What the Graph Portrays

At threshold 0.32, the model correctly detects 131 out of 156 genuine payments (True Positives) and correctly identifies 253 broken promises (True Negatives), with only 25 unmitigated missed payments (False Negatives).

7 The Unified End-to-End Pipeline

7.1 7.1 Architectural Overview of PTPPipeline

To ensure robust production deployment and eliminate brittle manual data manipulation, our entire data flow is encapsulated in a single in-memory class: PTPPipeline.

The pipeline executes sequentially in one invocation:

1. **Ingestion:** Reads accounts.csv, ptps.csv, payments.csv, call_transcripts.csv, and splits.csv.
2. **FIFO Digestion:** Computes exact promise attribution windows and consumes payments chronologically.
3. **Feature Synthesis:** Extracts speech kinetics, Indic NLP cues, circular payday offsets, and domain interaction terms.
4. **Leak-Free Splitting & Scaling:** Imputes missing values using training medians, generates unscaled matrices for tree algorithms, and standard-scales matrices for linear/neural models.

7.2 7.2 Easily Tweakable Hyperparameters

Top-level parameters can be modified directly at pipeline instantiation:

```
pipe = PTPPipeline(
    grace_days=2,          # Banking settlement grace days (0 to 5)
    theta=0.90,           # Fulfillment threshold ratio (0.50 to 1.00)
    random_state=42        # Reproducibility seed
)
data = pipe.fit_transform()
```

7.3 7.3 Code Snippets of Niche Methods

7.3.1 7.3.1 Deterministic FIFO Payment Attribution Loop

```
# Match payments to open promise windows chronologically per account
pay_by_acc = {a: g.sort_values('payment_ts') for a, g in payments.groupby('account_id')}
idx_by_acc = ptps.sort_values(['account_id', 'promised_date',
                               'captured_ts']).groupby('account_id').groups

paid_dict = {}
for acc_id, idxs in idx_by_acc.items():
    g = pay_by_acc.get(acc_id)
    if g is None:
        continue
    rem = {i: ptps.at[i, 'promised_amount'] for i in idxs}
    for i in idxs:
        paid_dict[i] = 0.0
    for r in g.itertuples():
        amt = r.amount
        for i in idxs:
            if ptps.at[i, 'captured_ts'] <= r.payment_ts <= ptps.at[i, 'window_end'] and rem[i] > 0:
                take = min(amt, rem[i])
                paid_dict[i] += take
                rem[i] -= take
                amt -= take
                if amt <= 0:
                    break
```

7.3.2 7.3.2 Circular Salary Day Proximity Formulation

```
# Models payday proximity across 30-day calendar boundary
p_day = m['promised_date'].dt.day
s_day = m['salary_credit_day'].fillna(-999)
diff = np.abs(p_day - s_day)
circ_diff = np.minimum(diff, 30 - diff)
m['feat_salary_day_diff'] = np.where(s_day > 0, circ_diff, 15.0)
m['feat_salary_proximity'] = np.where(s_day > 0, np.exp(-circ_diff / 3.0), 0.0)
```

7.3.3 7.3.3 Speech Hesitation & Kinetic Metrics

```
# Extracts cognitive hesitation and response latency from transcript turns
c_pauses = customer_rows['pause_before_ms'].dropna()
hesitation_rate = (c_pauses > 600).sum() / max(len(customer_rows), 1)
max_pause_ms = c_pauses.max() if len(c_pauses) > 0 else 0.0
```

7.3.4 7.3.4 Specialized PyTorch Focal Loss for Hard-Example Mining

```
class FocalLoss(nn.Module):
    def __init__(self, alpha=0.35, gamma=2.0):
        super().__init__()
        self.alpha = alpha
        self.gamma = gamma
    def forward(self, logits, targets):
        bce = nn.functional.binary_cross_entropy_with_logits(logits, targets, reduction='none')
        probs = torch.sigmoid(logits)
        p_t = targets * probs + (1 - targets) * (1 - probs)
        alpha_t = targets * self.alpha + (1 - targets) * (1 - self.alpha)
        loss = alpha_t * ((1 - p_t) ** self.gamma) * bce
        return loss.mean()
```

8 Strategic Recommendations & Conclusion

8.1 8.1 Summary of Key Findings

1. **Promise Lead Time is Decisive:** Promises scheduled beyond 4 days are overwhelmingly broken (> 85% failure rate).
2. **Acoustic & NLP Intent Outweighs Static History:** Customer speech pauses exceeding 600ms and conversational hedge phrases (“shayad”, “noddini”) provide powerful real-time evasion markers.
3. **Early Stopping Prevents Catastrophic Memorization:** For CatBoost, tree growth must be halted at Iteration 151; extending training to 400 rounds causes severe generalization decay (from 0.4766 up to 0.4896 loss).
4. **Probability Calibration is Essential:** Raw gradient booster margins are distorted; post-hoc isotonic calibration lowers Expected Calibration Error to 0.0354.
5. **Optimal Operating Cutoff is 0.32:** Operating at 0.32 instead of 0.50 maximizes net recovery value by preventing costly default escalations.

8.2 8.2 Operational Deployment Steps

- **Real-Time In-Call Scoring:** Deploy the champion model as a lightweight scoring microservice (< 15ms inference latency per call).
- **Automated Hold Window Adjustment:** If an in-call PTP receives a score < 0.32, do not place the borrower on a multi-day hold. Automatically trigger an SMS reminder within 24 hours.
- **Agent Workflow Optimization:** Prioritize manual human collector follow-up on accounts in the uncertain probability band (0.30 to 0.55), where an agent nudge creates the highest marginal impact.